Exercise Sheet 11

Show Gerald

House rules.

- First, inside the lesson folder: `pip install -r requirements.txt` (pandas, openpyxl, matplotlib).
- One file per exercise in `my_work/`. Everything you save goes to `my_work/output/`; create that folder at the top of every script with `Path("my_work/output").mkdir(parents=True, exist_ok=True)`.
- Every chart script starts with `import matplotlib` and `matplotlib.use("Agg")` **before** `import matplotlib.pyplot as plt`. Save with `plt.savefig(...)`; never `plt.show()`.
- Compare the printed output with the sheet, then **open the PNG or the workbook** and look at it. The file is the real result.
- Hand in: commit and push to `lesson-11`, open a pull request, and attach your favourite chart to it.

Exercise 1 Your first chart

10 min · `ex1_first_chart.py`

> **From Gerald (MD), Friday 16:55:** *"The board can't read tables. Can we have a picture of how many people we've got where? A bar thing. Blue, if possible. G."*

GIVEN

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
df = pd.read_csv("data/brightwell_payroll.csv")
```

(a) Count employees per department with `value_counts()`, biggest first, and print the result.

(b) Draw a bar chart of it (`plt.bar(counts.index, counts.values)`), with a title and a y-axis label. Rotate the department names 30° so they don't overlap: `plt.xticks(rotation=30, ha="right")`.

(c) Call `plt.tight_layout()`, then save as `my_work/output/headcount.png` at `dpi=150`. Open it.

EXPECTED OUTPUT

```
department
Production    60
Sales         35
Warehouse     25
IT            20
Finance       18
Marketing     15
Design        15
HR            12
Name: count, dtype: int64
Saved my_work/output/headcount.png
```

Hint: Gerald's blue is `color="#1f3a5f"`. Any colour name works too: `"steelblue"`, `"orange"`.

Exercise 2 Where do salaries sit?

10 min · `ex2_salary_histogram.py`

> **From Siobhan (Reward):** *"Before the pay review I want to see the salaries, not scroll through them. How many people earn roughly what? And where's the middle?"*

Same Given block as Exercise 1.

(a) Print the lowest, highest and median salary, formatted as pounds with thousands separators.

(b) Draw a histogram of `annual_salary` with `bins=12`. Add a dashed vertical line at the median: `plt.axvline(median, linestyle="--", label="Median")` and `plt.legend()`.

(c) Label both axes, add a title, save as `my_work/output/salaries.png`.

EXPECTED OUTPUT

```
Lowest salary:  £10,300
Highest salary: £68,750
Median salary:  £29,375
Saved my_work/output/salaries.png
```

Hint: A histogram is a bar chart of *how many* values fall into each bucket. Excel can do one, but only since 2016 and only if you find the button.

Exercise 3 Side by side

15 min · `ex3_side_by_side.py`

> **From Priya (Payroll):** *"Average salary per department, men next to women, one picture. The board meets in two weeks and I'd like them to notice something."*

Same Given block, plus `import numpy as np`.

(a) Build a pivot table: departments as rows, gender as columns, mean `annual_salary`, rounded to 0 decimals. Print it.

(b) Grouped bars: `x = np.arange(len(table.index))`, `width = 0.4`. Draw women at `x - width/2` and men at `x + width/2`, each with a `label=`. Put the department names on the ticks with `plt.xticks(x, table.index, rotation=30, ha="right")`.

(c) Y-axis label, title, legend, `figsize=(9, 4.5)`, save as `my_work/output/salary_by_dept_gender.png`.

(d) Look at the picture. Which department would you ask Priya about first?

EXPECTED OUTPUT

```
gender            F        M
department
Design      33594.0  32357.0
Finance     28267.0  36828.0
HR          27631.0  44125.0
IT          31264.0  45342.0
Marketing   29555.0  34638.0
Production  26372.0  30536.0
Sales       27150.0  35674.0
Warehouse   19075.0  34561.0
Saved my_work/output/salary_by_dept_gender.png
```

Exercise 4 Back to Excel

10 min · `ex4_first_workbook.py`

> **From Mo (IT):** *"Gerald's PA asked if the Python thing 'does Excel'. I said yes. Please make me right."*

GIVEN

```
import pandas as pd
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
df = pd.read_csv("data/brightwell_payroll.csv")
df["is_woman"] = df["gender"] == "F"
```

(a) Group by department and build a table with `employees` (count of `employee_id`), `women` (sum of `is_woman`) and `avg_salary` (mean, rounded to 0). Sort by employees, biggest first.

(b) Add a column `pct_women` = women / employees, rounded to 3 decimals. Print the table.

(c) Save it with `to_excel("my_work/output/headcount.xlsx", sheet_name="Headcount")`. Open the workbook.

EXPECTED OUTPUT

```
            employees  women  avg_salary  pct_women
department
Production         60     20     29148.0      0.333
Sales              35     14     32264.0      0.400
Warehouse          25      6     30844.0      0.240
IT                 20      7     40415.0      0.350
Finance            18      9     32547.0      0.500
Design             15      8     33017.0      0.533
Marketing          15     11     30910.0      0.733
HR                 12      8     33129.0      0.667
Saved my_work/output/headcount.xlsx
```

Hint: `True` counts as 1 when you sum it. That's why `is_woman` is handy.

Exercise 5 One workbook, several sheets

10 min · `ex5_several_sheets.py`

> **From Priya:** *"One file, please, not six. Tabs are fine. Tabs are great. Dave once sent me 14 workbooks called final.xlsx."*

Same Given block as Exercise 4 (without `is_woman`).

(a) Make three tables: salary `count`, `mean`, `median` by department; the same by location (both rounded to 0); and the top 10 earners with `employee_id, first_name, last_name, department, annual_salary`.

(b) Write all three into `my_work/output/salary_report.xlsx` using `with pd.ExcelWriter(path) as writer:` and one `to_excel(writer, sheet_name=...)` per table. The top 10 sheet gets `index=False`.

(c) Print the location table and a line naming the sheets.

EXPECTED OUTPUT

```
            count     mean   median
location
Leeds          67  33328.0  29950.0
London         40  30158.0  28250.0
Manchester     93  31834.0  30000.0
Sheets written: By department, By location, Top 10
```

Exercise 6 Make it pretty

15 min · `ex6_make_it_pretty.py`

> **From Gerald:** *"Lovely numbers. Could the top row be bold? And blue? And could the pounds look like pounds? And could the columns not be squashed? Thanks ever so. G."*

GIVEN

```
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill
from pathlib import Path

Path("my_work/output").mkdir(parents=True, exist_ok=True)
df = pd.read_csv("data/brightwell_payroll.csv")
```

(a) Per department: `employees`, `avg_salary` (mean), `total_bonus` (sum), sorted by average salary, highest first. Add `bonus_per_head` = total bonus / employees. Save to `my_work/output/pretty_report.xlsx`, sheet `Departments`.

(b) Re-open it with `load_workbook(path)`, pick the sheet with `wb["Departments"]`, and for every cell in row 1 set `Font(bold=True, color="FFFFFF")` and `PatternFill("solid", fgColor="1F3A5F")`.

(c) Give columns C to E the number format `"£#,##0.00"`: loop over `ws.iter_rows(min_row=2, min_col=3, max_col=5)`.

(d) Set column A to width 16 and B–E to 15 (`ws.column_dimensions["A"].width`), freeze the header with `ws.freeze_panes = "A2"`, save, and print the checks below.

EXPECTED OUTPUT

```
Rows: 9, columns: 5
A1 bold: True, C2 format: £#,##0.00
Saved my_work/output/pretty_report.xlsx
```

Hint: `ws.max_row`, `ws.max_column`, `ws["A1"].font.bold` and `ws["C2"].number_format` give you the four values to print.

Exercise 7 A chart inside the workbook STRETCH

15 min · `ex7_chart_in_excel.py`

> **From Gerald:** *"Can the picture live in the Excel? The board likes it when things are in the Excel."*

(a) Average salary per department, rounded, sorted smallest to largest. Print it. Draw a **horizontal** bar chart (`plt.barh`) and save it as `my_work/output/avg_salary.png`.

(b) Write the table to `my_work/output/report_with_chart.xlsx`, sheet `Data` (`avg.to_frame("avg_salary").to_excel(...)`).

(c) Re-open the workbook, `ws = wb.create_sheet("Chart")`, then `ws.add_image(Image("my_work/output/avg_salary.png"), "B2")` with `from openpyxl.drawing.image import Image`. Save. Print `wb.sheetnames`.

EXPECTED OUTPUT

```
department
Production    29148.0
Warehouse     30844.0
Marketing     30910.0
Sales         32264.0
Finance       32547.0
Design        33017.0
HR            33129.0
IT            40415.0
```

```
Name: annual_salary, dtype: float64
Sheets: ['Data', 'Chart']
Saved my_work/output/report_with_chart.xlsx
```

Bug hunt Dave's Leftovers #11

15 min · `bughunt/report_bug.py`

Dave's last project before the narrowboat: "the board pack in one click". Gerald clicked. Nothing happened. There are **three** bugs: one crashes, one shows the wrong people with a straight face, and one gives a number that's a bit too low (think about zeros).

EXPECTED OUTPUT ONCE FIXED

```
Top 3 earners:
EMP1129 Lucy Thompson 68750
EMP1144 Harry Walker 63650
EMP1112 Tom Robinson 61750
Average bonus of those who got one: £1,934.22
Saved my_work/output/board_pack.xlsx
```

One commit per bug, with a message that explains it.